

VITE & GOURMAND

Documentation technique

Architecture, données, sécurité, tests et déploiement

Projet ECF - TP Développeur Web et Web Mobile

Kelly FURNARI

Objet du document

Présenter les choix techniques et permettre à un développeur, un exploitant ou un membre du jury de comprendre, installer, tester, déployer et maintenir l'application Vite & Gourmand.

0 - Synthèse et repères

Vite & Gourmand est une application full-stack de traiteur. Elle propose une consultation publique des menus, la création de compte, la commande d'une prestation, le suivi des statuts, les avis, ainsi que des espaces employés et administrateur.

L'architecture sépare clairement l'interface Angular, l'API Spring Boot, les données transactionnelles PostgreSQL et les statistiques MongoDB. Les e-mails sont capturés par Mailpit en local et envoyés par Brevo en production.

Élément	Choix appliqué
Frontend	Angular 20 standalone, TypeScript, HTML5 et SCSS
Backend	Java 21 LTS et Spring Boot 3.5.4
Sécurité	Spring Security, JWT, refresh token, BCrypt et rôles
SQL	PostgreSQL 16, JPA/Hibernate et Flyway V1 à V3
NoSQL	MongoDB 7 pour les statistiques agrégées
Local	Docker Compose avec PostgreSQL, MongoDB et Mailpit
Production	Render, Neon, MongoDB Atlas et Brevo
Qualité	64 tests backend, 55 tests Angular, builds et recette

Liens du projet

[Dépôt GitHub public](#) / [Application](#) / [API](#)

1 - Réflexion initiale et choix technologiques

1.1 Contraintes du sujet

Le sujet impose une application web sécurisée, responsive et déployée, avec une base relationnelle et une base non relationnelle. Il demande également plusieurs rôles, des parcours métier complets, des e-mails, une documentation et des diagrammes.

1.2 Choix et justification

Technologie	Pourquoi ce choix	Alternative envisagée
Angular 20 standalone	Structure en composants, formulaires, services et routes ; application monopage dynamique et testable.	JavaScript sans framework aurait demandé davantage de structure manuelle.
Java 21 + Spring Boot 3.5.4	Écosystème robuste pour REST, validation, sécurité, transactions, JPA et tests.	Node.js aurait été possible, mais Spring Boot fournit un cadre métier très structuré.
PostgreSQL 16	Contraintes, relations, transactions et calculs fiables pour commandes et stock.	MySQL/MariaDB convenaient aussi, mais PostgreSQL est riche et bien supporté.
MongoDB 7	Stockage souple des agrégats statistiques sans dupliquer le modèle transactionnel.	Un second schéma SQL aurait moins clairement répondu à l'exigence NoSQL.
Docker Compose	Environnement local reproductible pour PostgreSQL, MongoDB et Mailpit.	Installations locales séparées plus difficiles à reproduire.
Render + services managés	Déploiement public simple, HTTPS et variables d'environnement.	Un VPS aurait demandé davantage d'administration système.

Décision structurante

La base SQL reste la source de vérité pour les données métier. MongoDB contient uniquement des statistiques dérivées, donc reconstituables en cas de besoin.

L'application suit une architecture client-serveur. Angular est livré comme site statique, tandis que Spring Boot expose une API REST. Les échanges sont réalisés en JSON via HTTPS et protégés par des jetons Bearer JWT.

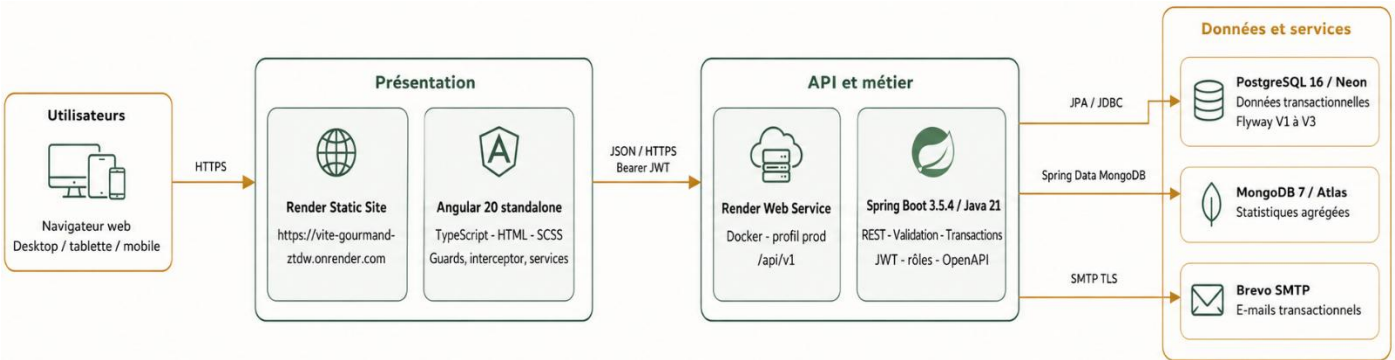


Figure 1 - Architecture de production

2.1 Responsabilités

Couche	Responsabilité
Angular	Affichage, formulaires, navigation, guards, interceptor JWT et appels HTTP.
API Spring Boot	Validation, règles métier, sécurité, transactions, persistance et e-mails.
PostgreSQL	Utilisateurs, catalogue, commandes, historiques, avis, horaires et jetons.
MongoDB	Statistiques agrégées par menu pour le tableau de bord administrateur.
Brevo	Envoi SMTP des messages transactionnels.
Render	Hébergement du front-end statique et du back-end Dockerisé.

3 - Configuration de l'environnement de travail

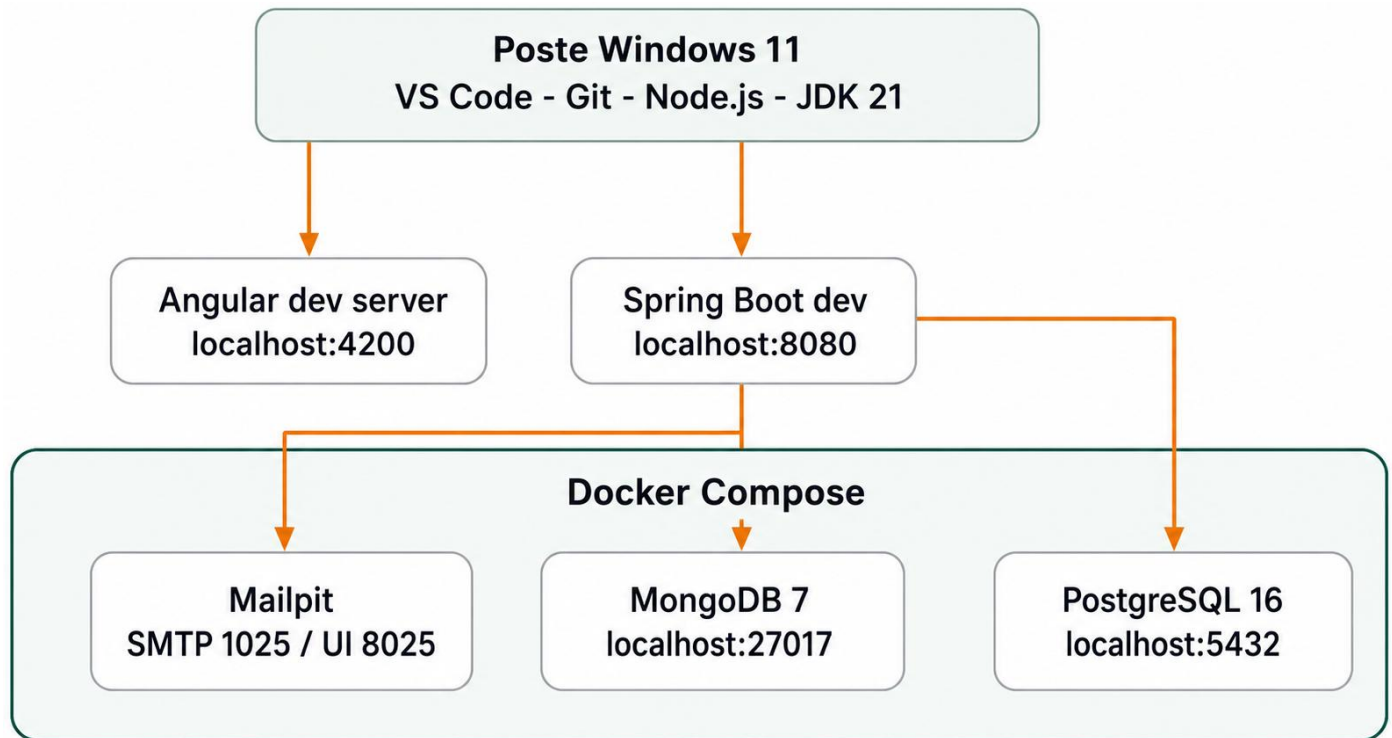


Figure 2 - Environnement de développement local

3.1 Prérequis

- Windows 11 ou un système équivalent
- Git et un compte GitHub
- Eclipse Temurin JDK 21
- Node.js et npm
- Docker Desktop avec Docker Compose
- Visual Studio Code ou un IDE équivalent

3.2 Démarrage local

```
docker compose up -d
docker compose ps

cd backend
$env:SPRING_PROFILES_ACTIVE = "dev"
.\mvnw.cmd spring-boot:run

cd ..\frontend
npm install
npm start
```

Service	Adresse / port	Usage
Angular	http://localhost:4200	Interface de développement
API	http://localhost:8080	API REST Spring Boot
Swagger	http://localhost:8080/swagger-ui/index.html	Documentation interactive
PostgreSQL	localhost:5432	Base relationnelle
MongoDB	localhost:27017	Base NoSQL
Mailpit	http://localhost:8025	Consultation des e-mails locaux

Reproductibilité

Le Maven Wrapper et le fichier package-lock.json réduisent les différences de versions. Les conteneurs évitent d'installer PostgreSQL, MongoDB et Mailpit séparément.

4 - Organisation du code

4.1 Structure du dépôt

```
vite-et-gourmand/  
├── backend/      API Spring Boot, migrations et tests Java  
├── frontend/     application Angular et tests  
├── database/     schema.sql et data.sql  
├── documentation/ livrables, diagrammes et documents  
├── scripts/      smoke tests et nettoyage  
├── docker-compose.yml  
├── render.yaml  
└── README.md
```

4.2 Organisation du back-end

Le back-end est découpé par domaines : authentification, utilisateurs, catalogue, menus, plats, allergènes, commandes, avis, statistiques, contact et configuration. Chaque domaine utilise selon les besoins des contrôleurs, DTO, services, entités ou documents, et repositories.

4.3 Organisation du front-end

Le front-end sépare les éléments transverses dans core (authentification, modèles et services) et les écrans dans features. Les templates HTML et les styles spécifiques ont été externalisés afin de rendre le TypeScript plus lisible.

Élément Angular	Rôle
Routes	Associent les URL aux composants et appliquent les guards.
Services	Centralisent les appels HTTP et la logique de session.
Interceptor	Ajoute le jeton d'accès aux requêtes protégées.
Guards	Améliorent la navigation selon le rôle connecté.
Composants	Gèrent l'état d'écran et les formulaires.
HTML / SCSS	Séparent la vue et la présentation du code TypeScript.

5 - Modèle de données relationnel

PostgreSQL contient les données transactionnelles et garantit leur cohérence par les clés étrangères, les contraintes, les transactions et les migrations Flyway. Le schéma peut également être recréé à partir des fichiers SQL présents dans database/.

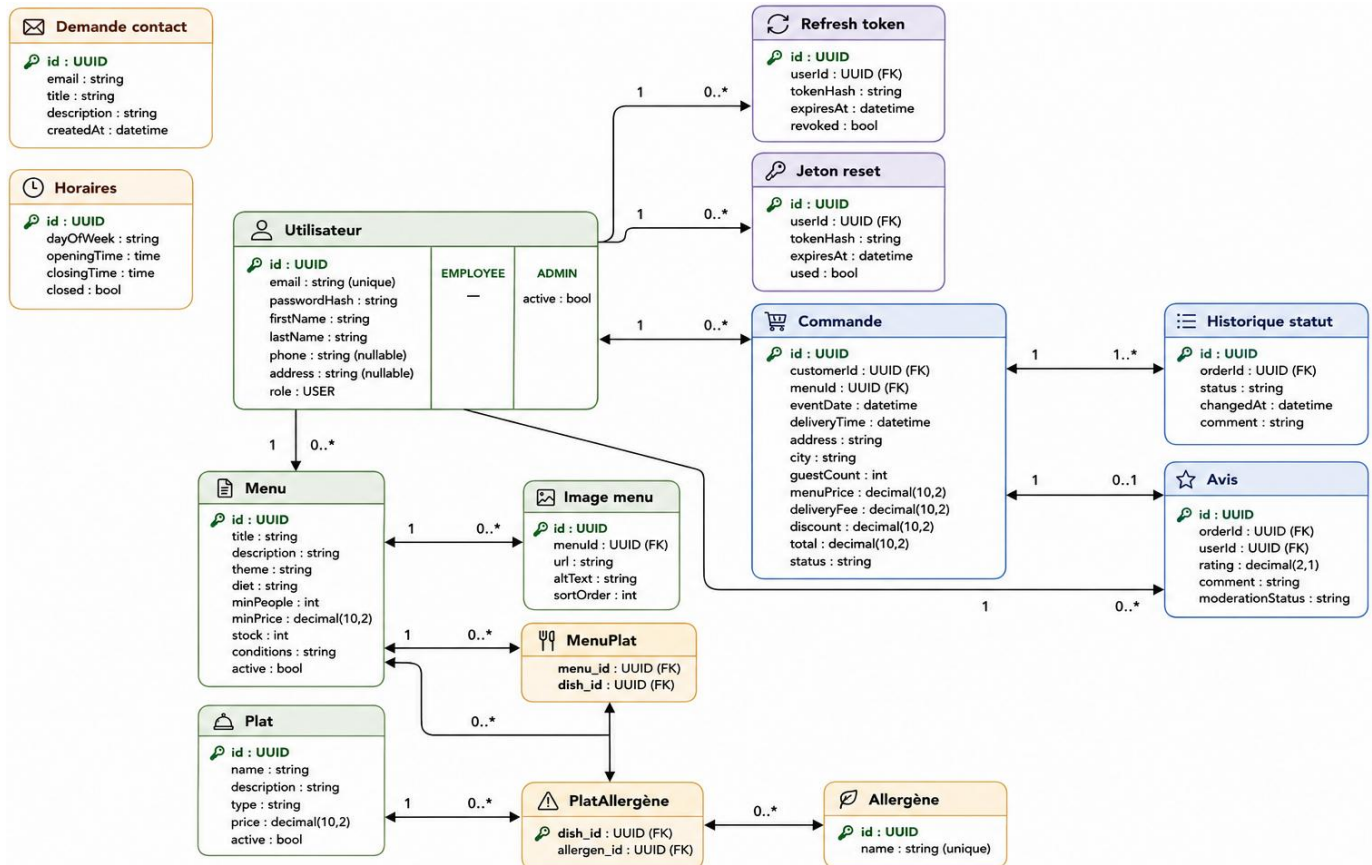


Figure 3 - Modèle relationnel simplifié

5.1 Relations principales

- Un utilisateur peut passer plusieurs commandes.
- Une commande concerne un menu et possède un historique de statuts.
- Un menu propose plusieurs plats et un plat peut être réutilisé dans plusieurs menus.
- Un plat peut contenir plusieurs allergènes.
- Une commande terminée peut recevoir un avis unique, soumis à modération.
- Les refresh tokens et les jetons de réinitialisation sont rattachés à un utilisateur.

Intégrité métier

La création d'une commande, la décrémentement du stock et l'écriture de l'historique initial sont réalisées dans une même transaction. Une erreur annule l'ensemble des écritures.

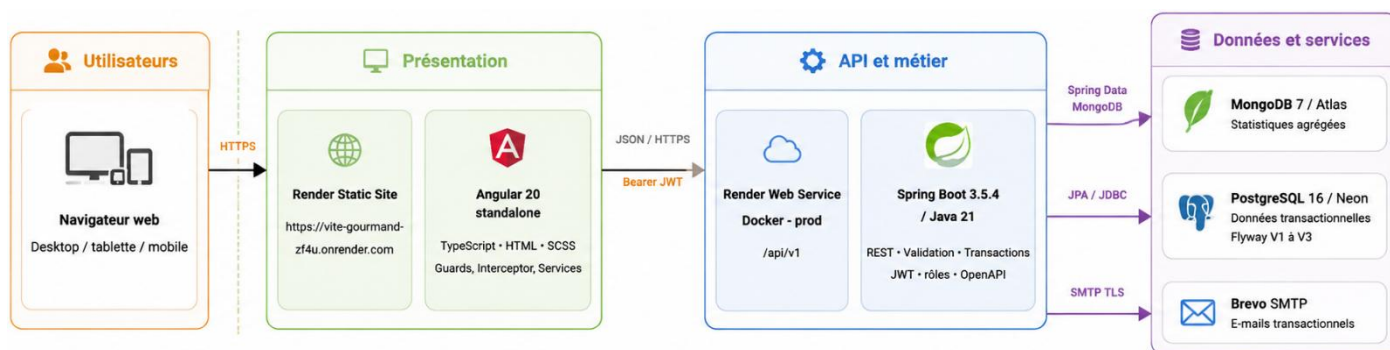


Figure 4 - Construction des statistiques MongoDB

MongoDB n'est pas utilisé pour remplacer PostgreSQL. Il stocke des documents agrégés permettant à l'administrateur de comparer le nombre de commandes et le chiffre d'affaires par menu.

Champ du document	Description
menuId	Identifiant du menu source.
menuTitle	Titre affiché dans le tableau de bord.
orderCount	Nombre de commandes prises en compte.
revenue	Chiffre d'affaires agrégé.
updatedAt	Date de la dernière reconstruction.

La reconstruction reste répétable : les statistiques peuvent être recalculées depuis les commandes SQL, ce qui limite le risque de divergence durable.

7 - Diagramme de cas d'utilisation

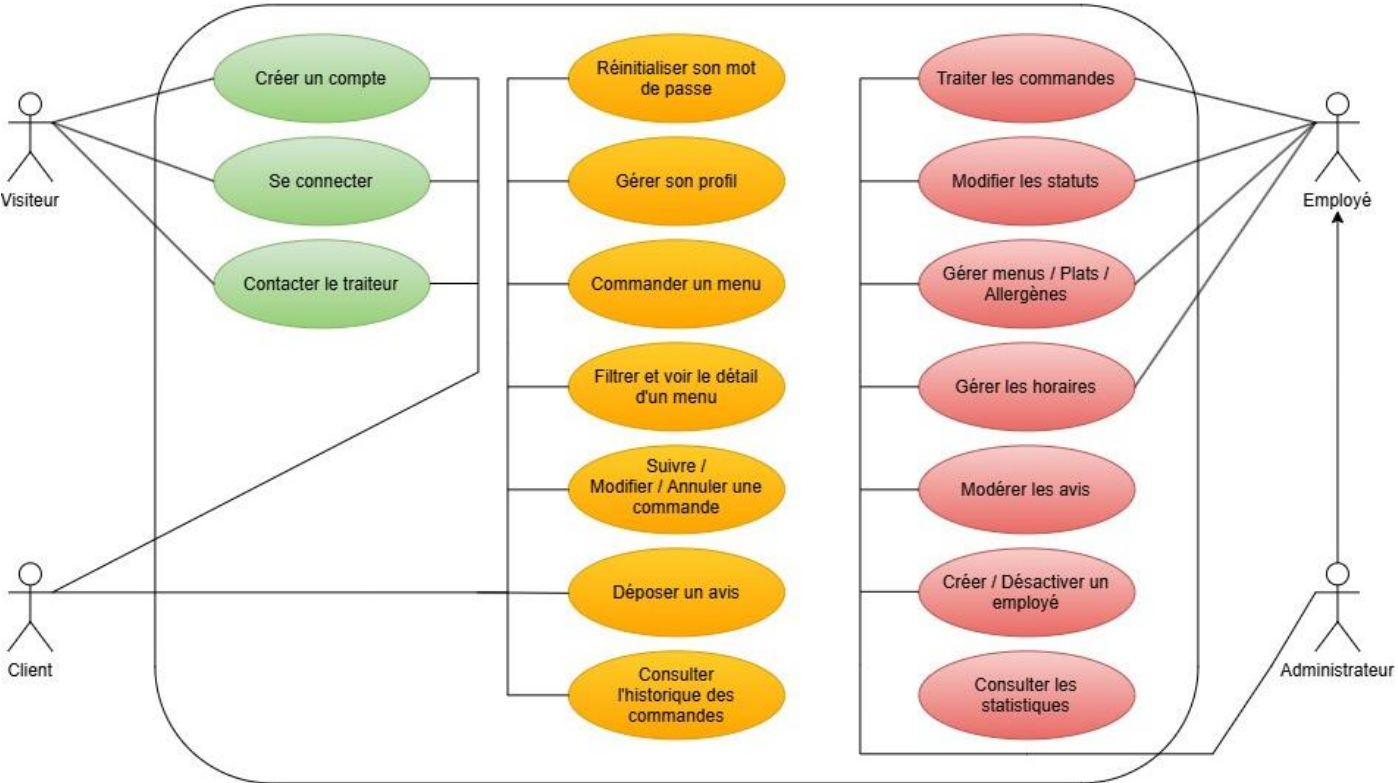


Figure 5 - Cas d'utilisation par rôle

Acteur	Principaux droits
Visiteur	Consulter les menus, filtrer, voir les détails, contacter, créer un compte et se connecter.
Client	Gérer son profil, commander, suivre ou annuler selon le statut, puis déposer un avis.
Employé	Traiter les commandes, modifier les statuts et gérer le catalogue, les horaires et les avis.
Administrateur	Dispose des droits employés, gère les comptes employés et consulte les statistiques.

Principe d'autorisation

Les restrictions d'interface facilitent le parcours, mais la décision de sécurité est toujours appliquée par Spring Security et les services back-end.

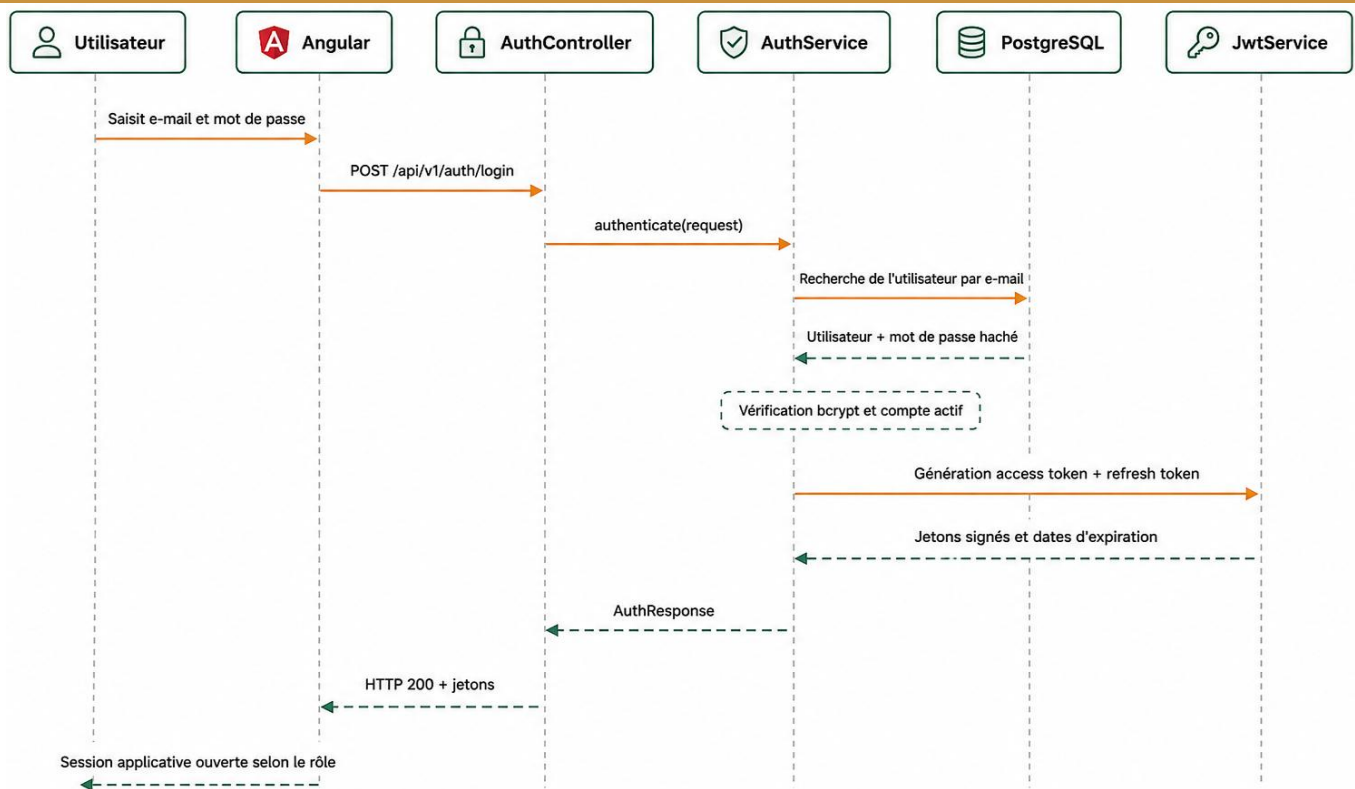


Figure 6 - Connexion avec access token et refresh token

Le mot de passe n'est jamais comparé en clair. Spring Security vérifie le hash BCrypt, contrôle l'état actif du compte, puis l'API génère un jeton d'accès de courte durée et un refresh token révoquant.

8.1 Renouvellement et déconnexion

- Le front-end utilise le refresh token pour obtenir un nouveau jeton d'accès lorsque nécessaire.
- La déconnexion révoque le refresh token côté serveur.
- Un refresh token révoqué ou expiré provoque une réponse 401.
- Le mot de passe oublié utilise une réponse neutre et un jeton temporaire à usage unique.

9 - Séquence de création d'une commande

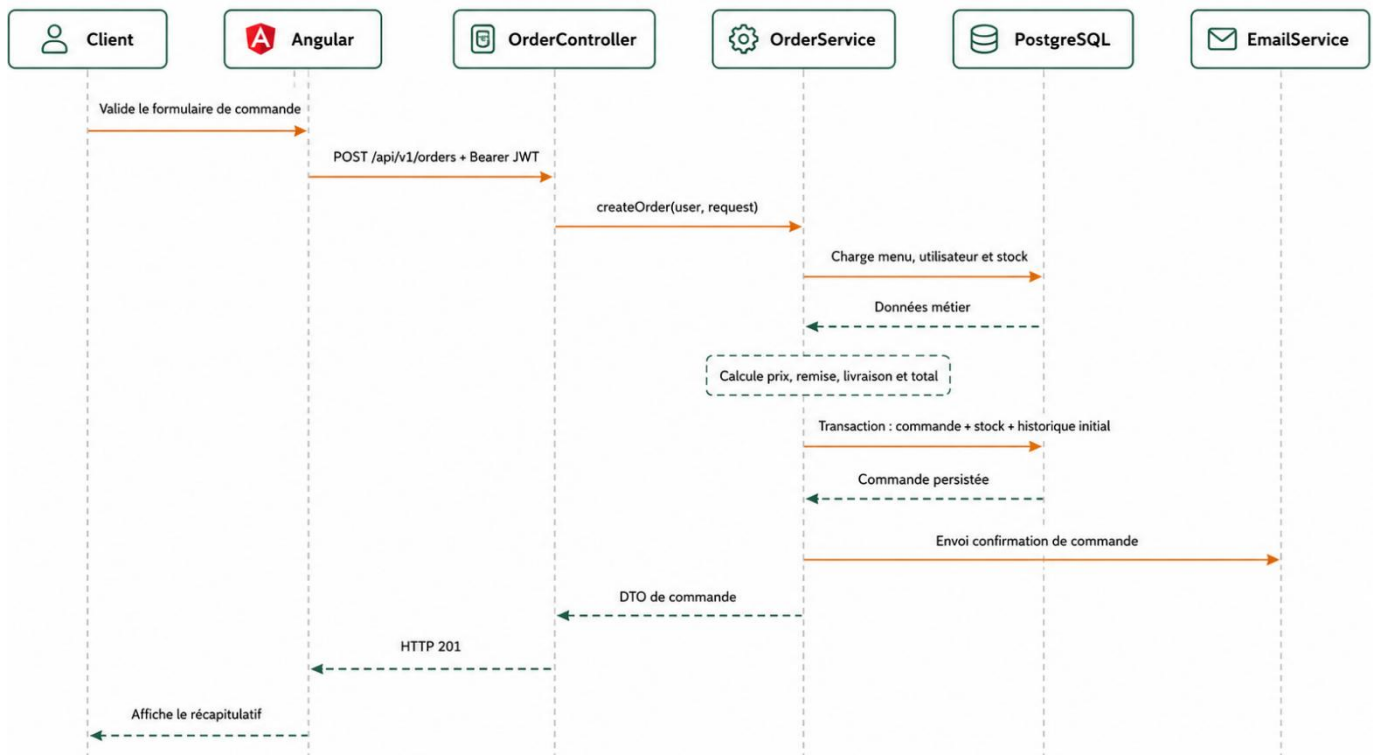


Figure 7 - Création transactionnelle d'une commande

9.1 Calculs exécutés côté serveur

- Respect du nombre minimal de personnes ;
- Calcul du prix du menu selon le nombre de convives ;
- Remise de 10 % à partir de cinq personnes au-dessus du minimum ;
- Frais de livraison hors Bordeaux : 5 euros plus 0,59 euro par kilomètre ;
- Contrôle du stock et du statut actif du menu ;
- Création de l'historique initial et envoi d'une confirmation.

Le frontend affiche un aperçu, mais l'API recalcule systématiquement le prix afin d'éviter qu'un client puisse modifier le montant transmis.

10 - Séquence de traitement d'une commande

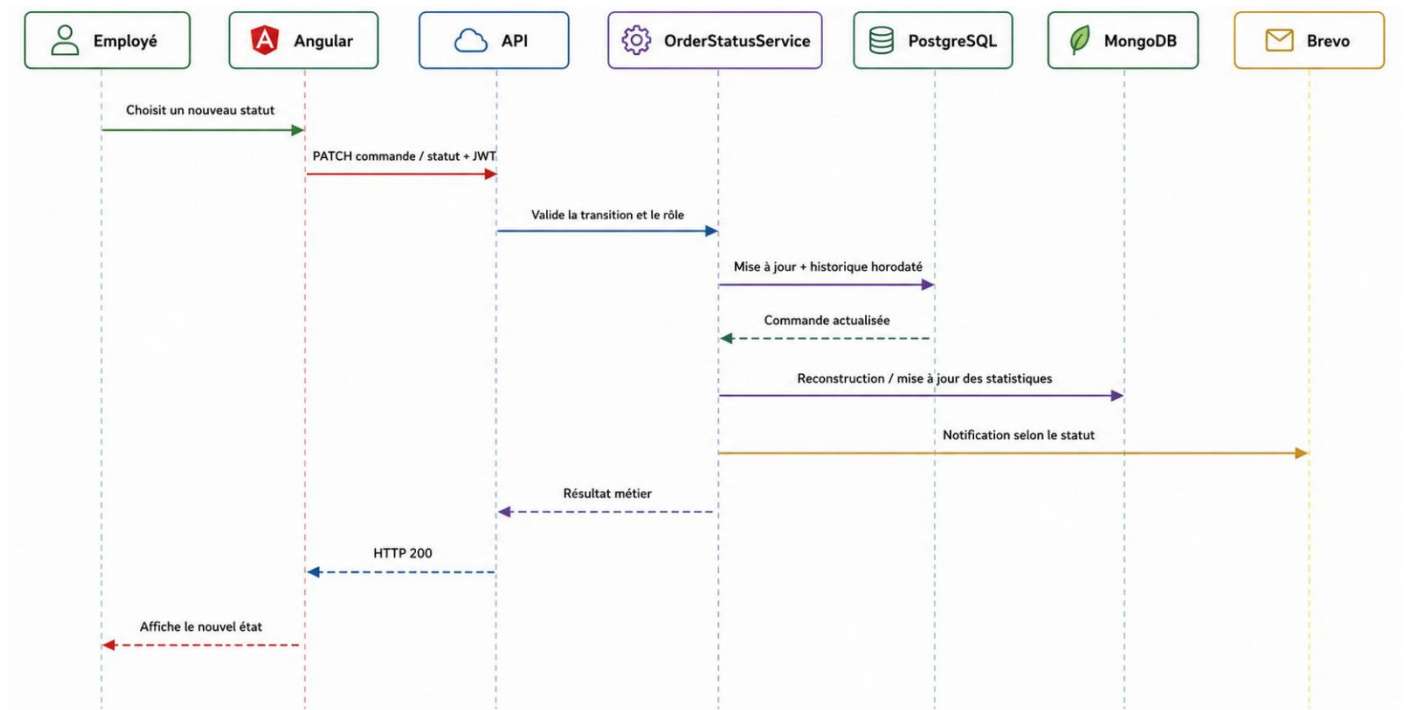


Figure 8 - Changement de statut, statistiques et notification

L'employé ou l'administrateur sélectionne un nouveau statut. Le serveur contrôle la transition autorisée, écrit une ligne d'historique horodatée, met à jour les agrégats et déclenche les notifications prévues.

Statut	Signification
PENDING	Commande enregistrée en attente de validation.
ACCEPTED	Commande acceptée par l'équipe.
PREPARING	Préparation en cuisine.
DELIVERING	Livraison en cours.
DELIVERED	Prestation livrée.
WAITING_FOR_EQUIPMENT_RETURN	Matériel à restituer.
COMPLETED	Commande terminée, avis possible.
CANCELLED	Commande annulée avec motif et mode de contact si nécessaire.

11 - Sécurité

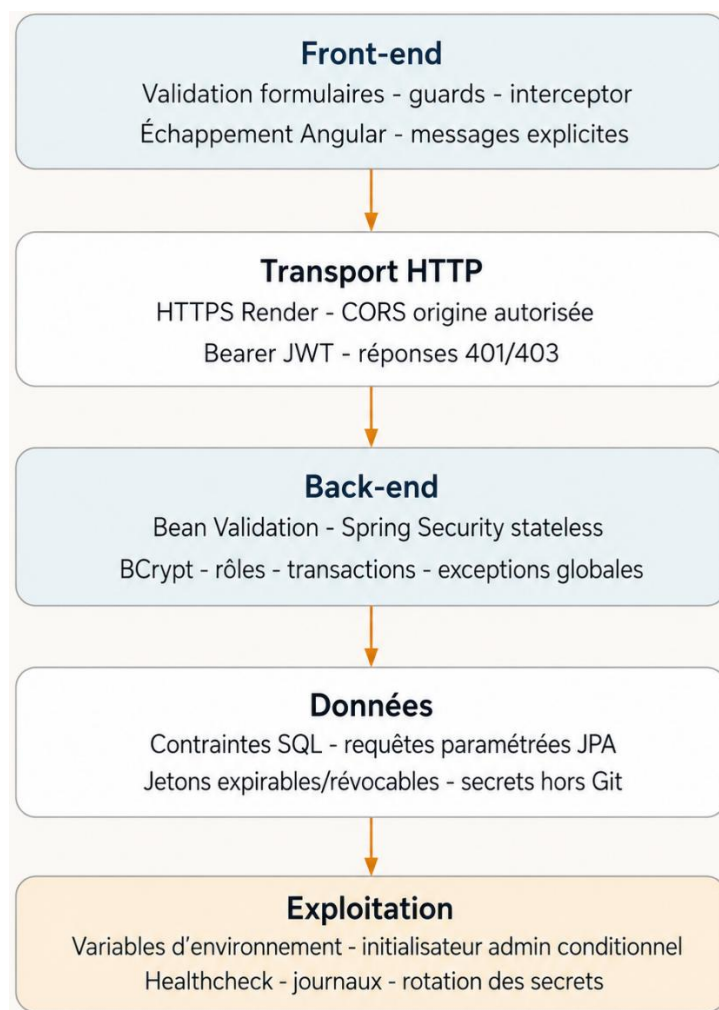


Figure 9 - Défense en profondeur

11.1 Formulaires et entrées

- Validation côté Angular pour guider l'utilisateur.
- Bean Validation côté Spring pour refuser les données invalides.
- Mot de passe : au moins 10 caractères avec majuscule, minuscule, chiffre et caractère spécial.
- Messages d'erreur explicites, sans détail technique sensible.

11.2 Authentification et autorisation

- BCrypt pour le hachage des mots de passe.
- Session HTTP stateless avec JWT.
- Refresh tokens persistés, expirables et révocables.
- Rôles USER, EMPLOYEE et ADMIN contrôlés sur les routes et les opérations métier.
- Réponses 401 pour l'absence d'authentification et 403 pour un rôle insuffisant.

11.3 Données et exploitation

- JPA/Hibernate produit des requêtes paramétrées.
- Contraintes SQL et transactions protègent l'intégrité.
- CORS autorise uniquement l'origine front-end configurée.
- Secrets conservés dans les variables d'environnement et fichiers ignorés par Git.
- HTTPS fourni par les services Render.
- Initialiseur administrateur désactivé après la création du compte.

Limite connue

Aucune sécurité front-end ne remplace les contrôles serveur. Un utilisateur peut modifier son navigateur ; l'API doit donc toujours vérifier l'identité, le rôle, la propriété de la ressource et les règles métier.

12 - API REST et documentation OpenAPI

Toutes les routes sont préfixées par /api/v1. Les contrôleurs utilisent des DTO afin de ne pas exposer directement les entités persistées.

Famille	Exemples	Accès
Public	GET menus, détail, horaires, avis ; POST contact ; GET health	Sans authentification
Authentification	register, login, refresh, logout, forgot-password, reset-password	Public avec contrôles
Profil	GET/PUT utilisateur courant	USER / EMPLOYEE / ADMIN
Commandes	création, liste, détail, modification, annulation	Client propriétaire
Employé	liste filtrée, détail, statuts et gestion métier	EMPLOYEE / ADMIN
Administration	employés, catalogue, horaires, avis et statistiques	ADMIN selon la fonction

12.1 Contrats HTTP vérifiés

Appel	Résultat attendu
POST /api/v1/auth/register	201
POST /api/v1/auth/login	200
POST /api/v1/auth/refresh	200
POST /api/v1/auth/logout	204
GET /api/v1/users/me sans JWT	401
Route ADMIN avec rôle USER	403
POST /api/v1/orders	201
POST /api/v1/public/contact	202
GET /api/v1/public/health	200

Swagger UI est disponible en local sur <http://localhost:8080/swagger-ui/index.html>. Le document OpenAPI expose un schéma bearerAuth de type HTTP Bearer.

Les valeurs réelles ne doivent jamais être inscrites dans le dépôt. Le fichier .env.example ne contient que des exemples et le fichier .env est ignoré par Git.

Variable	Rôle	Sensibilité
SPRING_PROFILES_ACTIVE	Choix du profil dev ou prod.	Faible
DATABASE_URL / configuration JDBC	Connexion PostgreSQL Neon.	Élevée
POSTGRES_USER / POSTGRES_PASSWORD	Identifiants PostgreSQL local ou distant.	Élevée
MONGODB_URI	Connexion MongoDB locale ou Atlas.	Élevée
JWT_SECRET	Signature des jetons JWT.	Critique
FRONTEND_URL	Origine autorisée par CORS.	Faible
API_URL	URL de l'API injectée dans le front-end.	Faible
MAIL_HOST / MAIL_PORT	Serveur SMTP.	Faible
MAIL_USERNAME / MAIL_PASSWORD	Authentification Brevo.	Critique

Règle opérationnelle

Après la première connexion administrateur réussie, INITIAL_ADMIN_ENABLED doit être désactivé afin d'éviter toute recréation involontaire.

Zone	Commande / outil	Résultat constaté
Backend	mvnw clean test	64 tests réussis
Backend	mvnw clean package -DskipTests	JAR Spring Boot généré
Frontend	tests Angular	55 tests réussis
Frontend	build de production	Build réussi
E2E	Playwright / Chromium	Parcours ciblés validés
Git	git diff --check	Aucune erreur d'espacement
API	Swagger, CORS, 401/403 et smoke tests	Contrôles réussis
E-mails	Mailpit puis Brevo	Réception vérifiée
Production	healthcheck et recette des rôles	Application fonctionnelle

14.1 Couverture fonctionnelle

- Inscription, normalisation de l'e-mail et hachage ;
- Connexion, refresh, révocation et déconnexion ;
- Mot de passe oublié et jetons expirés ;
- Calcul du prix, minimum, remise et livraison ;
- Stock, création transactionnelle et historique ;
- Modification, annulation et transitions de statut ;
- Avis, modération et visibilité publique ;
- Statistiques restructurables ;
- Guard, interceptor et formulaires Angular ;
- CORS, Swagger, Docker, Flyway et e-mails.

Avertissement non bloquant

Mockito/ByteBuddy signale un futur changement de chargement d'agent avec les prochaines versions du JDK. Les tests réussissent actuellement ; une mise à jour de configuration sera à prévoir lors d'une future montée de version.

15 - Déploiement de l'application

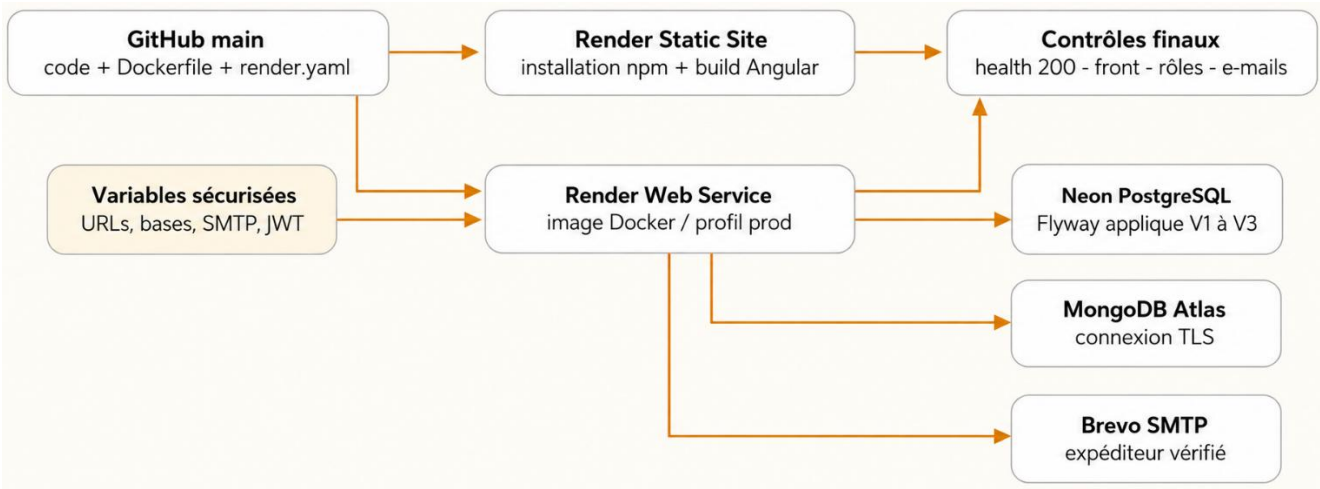


Figure 10 - Chaîne de déploiement

15.1 Front-end Render

1. Créer un Static Site relié au dépôt GitHub.
2. Définir le dossier frontend comme racine de build.
3. Installer les dépendances et produire le build Angular.
4. Injecter API_URL dans la configuration runtime.
5. Publier le dossier généré et configurer la redirection SPA.

15.2 Back-end Render

1. Créer un Web Service relié au même dépôt.
2. Construire l'image avec backend/Dockerfile.
3. Activer le profil prod.
4. Configurer PostgreSQL Neon, MongoDB Atlas, Brevo, JWT et CORS.
5. Laisser Flyway appliquer les migrations au démarrage.
6. Contrôler /api/v1/public/health puis les journaux.

15.3 Services externes

Service	Configuration essentielle
Neon	Base PostgreSQL 16, utilisateur dédié, URL JDBC sécurisée.
MongoDB Atlas	Cluster, utilisateur, liste d'accès réseau et URI mongodb+srv.
Brevo	Clé SMTP, expéditeur vérifié et adresse de contact.
Render	Variables d'environnement, URL front-end, healthcheck et redéploiement.

16 - Recette de production et exploitation

16.1 Contrôles après déploiement

- Le frontend se charge après un Ctrl + F5 et appelle la bonne API.
- Le healthcheck répond HTTP 200 avec un statut UP.
- Un client peut s'inscrire, se connecter, commander et utiliser le mot de passe oublié.
- Un employé peut traiter une commande et modifier son statut.
- Un administrateur peut gérer le catalogue, les employés, les horaires et les statistiques.
- Le formulaire de contact et les e-mails de réinitialisation apparaissent dans Brevo.
- Aucun secret réel ni stack trace n'est visible dans les réponses ou le dépôt.

16.2 Diagnostic

Symptôme	Vérification
Front sans données	Contrôler API_URL, CORS, journaux navigateur et disponibilité de l'API.
API indisponible	Consulter les logs Render, le profil actif et le healthcheck.
Erreur SQL	Contrôler DATABASE_URL, l'accès Neon et les migrations Flyway.
Erreur MongoDB	Contrôler MONGODB_URI, DNS SRV et liste d'accès Atlas.
E-mail absent	Contrôler Brevo, l'expéditeur, les identifiants SMTP et les spams.
401 / 403	Vérifier jeton, expiration, refresh token, rôle et propriété de la ressource.

Secrets

En cas d'exposition accidentelle, la valeur doit être révoquée ou remplacée immédiatement dans le fournisseur concerné, puis mise à jour dans Render. Supprimer seulement le texte dans Git ne rend pas l'ancien secret sûr.

17 - Maintenance, sauvegarde et évolutions

17.1 Évolution de la base

- Ne jamais modifier une migration Flyway déjà appliquée.
- Créer une migration suivante avec un nom explicite.
- Tester la migration sur une base locale avant production.
- Conserver schema.sql et data.sql comme preuve de maîtrise SQL.

17.2 Mise à jour applicative

- Créer une branche depuis develop.
- Limiter le périmètre et ajouter les tests concernés.
- Exécuter tests, builds et recette locale.
- Fusionner dans develop, contrôler l'intégration, puis publier dans main.
- Vérifier Render et les services externes après le déploiement.

17.3 Axes d'amélioration

Amélioration	Bénéfice
GitHub Actions	Tests et builds automatiques à chaque push ou pull request.
Tests E2E complets	Réduction des régressions sur les parcours client, employé et admin.
Audit RGAA automatisé	Suivi continu de l'accessibilité.
Sauvegarde/restauration documentée	Réduction du risque de perte de données.
Métriques et alertes	Détection plus rapide des pannes ou erreurs.
Service de géocodage	Calcul de distance plus fiable hors Bordeaux.

18 - Références et checklist finale

18.1 Documents du dépôt

Ressource	Emplacement
README	README.md
Scripts SQL	database/schema.sql et database/data.sql
Migrations	backend/src/main/resources/db/migration/
Manuel utilisateur	documentation/manuel-utilisateur/
Charte graphique	documentation/charte-graphique/
Gestion de projet	documentation/gestion-projet/
Documentation technique	documentation/documentation-technique/
Diagrammes	documentation/diagrammes/

18.2 Références techniques

Angular : <https://angular.dev>

Spring Boot : <https://spring.io/projects/spring-boot>

Spring Security : <https://spring.io/projects/spring-security>

PostgreSQL : <https://www.postgresql.org/docs/>

MongoDB : <https://www.mongodb.com/docs/>

Flyway : <https://documentation.red-gate.com/flyway>

Render : <https://render.com/docs>

Neon : <https://neon.com/docs>

Brevo : <https://developers.brevo.com/docs>

OWASP : <https://owasp.org/www-project-top-ten/>

18.3 Checklist de remise

- ☐ Dépôt public et branches main/develop à jour.
- ☐ Front-end et API accessibles.
- ☐ README et installation locale vérifiés.
- ☐ schema.sql et data.sql présents.
- ☐ Manuel utilisateur PDF présent et comptes de démonstration testée.
- ☐ Charte graphique PDF avec 3 maquettes bureautiques et 3 mobiles.

- ☐ Documentation de gestion de projet présente.
- ☐ Documentation technique et diagrammes présents.
- ☐ Aucun secret réel dans Git.
- ☐ Copie ECF complétée avec liens et identifiants.

Conclusion

L'architecture retenue répond aux exigences fonctionnelles et techniques de l'ECF. Elle sépare les responsabilités, applique les règles métier côté serveur, protège les accès par rôle et fournit une chaîne complète de développement, de test et de déploiement.